

## CSCE 313

Introduction to Computer Systems

Texas A&M University • Fall 2026

# LAB 4

## Signals: Interrupts, Timeouts & Child Processes

---

### Student Companion Guide

Step-by-step walkthrough, explanations,  
and the reasoning behind every command

This guide accompanies the official lab handout. Where the two differ,  
the handout governs the grade — but read the notes here first.

October 8, 2026

# Contents

|          |                                                                       |           |
|----------|-----------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Start Here</b>                                                     | <b>3</b>  |
| 1.1      | How to use this guide . . . . .                                       | 3         |
| 1.2      | The big picture . . . . .                                             | 3         |
| 1.3      | What you edit, and how it is graded . . . . .                         | 4         |
| 1.4      | Build and test loop . . . . .                                         | 5         |
| <b>2</b> | <b>Signals in Fifteen Minutes</b>                                     | <b>7</b>  |
| 2.1      | Idea 1: a signal is a software interrupt . . . . .                    | 7         |
| 2.2      | Idea 2: pending, blocked, delivered . . . . .                         | 7         |
| 2.3      | Idea 3: signals interrupt system calls . . . . .                      | 8         |
| 2.4      | Idea 4: a handler may only call async-signal-safe functions . . . . . | 9         |
| 2.5      | Idea 5: share state with a handler through atomic flags . . . . .     | 9         |
| <b>3</b> | <b>Part A — signals.cpp</b>                                           | <b>11</b> |
| 3.1      | Task 1 — Define the three atomic flags . . . . .                      | 11        |
| 3.2      | Task 2 — Install the three handlers with sigaction . . . . .          | 13        |
| 3.3      | Task 3 — The SIGINT handler: flag first, forced exit second . . . . . | 15        |
| 3.4      | Task 4 — The SIGALRM handler . . . . .                                | 17        |
| 3.5      | Task 5 — The SIGCHLD handler: reap, mark inactive, record . . . . .   | 19        |
| 3.6      | Task 6 — block_signals() and unblock_signals() . . . . .              | 20        |
| 3.7      | Task 7 — wait_with_timeout() and cancel_timeout() . . . . .           | 23        |
| <b>4</b> | <b>Part B — client.cpp</b>                                            | <b>25</b> |
| 4.1      | Task 8 — Register the three servers . . . . .                         | 25        |
| 4.2      | Task 9 — Protect every transaction with block/unblock . . . . .       | 27        |
| <b>5</b> | <b>Prove It, Then Submit</b>                                          | <b>30</b> |
| 5.1      | Experiment 1: a normal session . . . . .                              | 30        |
| 5.2      | Experiment 2: Ctrl+C at the menu (graceful shutdown) . . . . .        | 31        |
| 5.3      | Experiment 3: Ctrl+C during a transaction (deferred) . . . . .        | 31        |
| 5.4      | Experiment 4: the second Ctrl+C (forced exit) . . . . .               | 32        |

---

|     |                                              |    |
|-----|----------------------------------------------|----|
| 5.5 | Things that look wrong but are not . . . . . | 33 |
| 5.6 | Troubleshooting by grader message . . . . .  | 33 |
| 5.7 | Before you submit . . . . .                  | 35 |
| 5.8 | Quick reference . . . . .                    | 36 |

# Start Here

## 1.1 How to use this guide

This guide walks you through Lab 4 one task at a time. It does **not** give you the code for the TODOs. You write that yourself. What it gives you is the mechanism behind every TODO, the system calls you need and how they behave, hints and pseudo-code, the exact checks the autograder runs, and the mistakes that cost students points.

Every task follows the same seven boxes, in the same order. The colour tells you what kind of box you are looking at:

|                  |                                                                       |
|------------------|-----------------------------------------------------------------------|
| Commands         | The system calls and types this task uses.                            |
| Task             | What the TODO asks, in plain language, and how it is graded.          |
| How to solve     | Hints, API shapes and pseudo-code. The thinking, not the answer.      |
| What's happening | The mechanism underneath. Read this one even if you already finished. |
| Real-world use   | Where this exact idea shows up outside the course.                    |
| Intuition        | The one-paragraph mental model to keep.                               |
| Takeaway         | The summary. Skim only these and you can rebuild the lab.             |

Four support boxes appear wherever they are needed: **red** boxes are mistakes that cost points, grey notes are “this looks wrong but is expected”, **blue** tips are optional depth, and **green** checkpoints tell you how to confirm a step before moving on.

### This guide and the handout

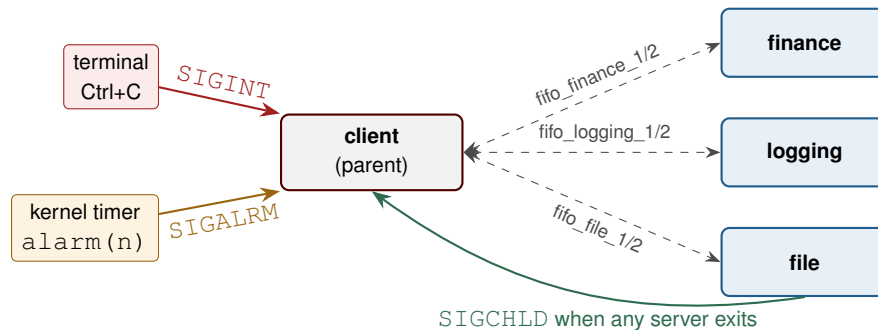
This guide accompanies the official handout; it does not replace it. If the two ever disagree, the handout and the autograder decide your grade. Tell a TA when you find a disagreement, so the guide can be fixed.

## 1.2 The big picture

You already know this program. It is the banking system from the previous labs: a `client` that forks three servers (`finance`, `logging` and `file`), `execs` them, and talks to each over a pair of named pipes (FIFOs). Lab 4 adds the missing piece: what happens when something *interrupts* the program.

Three different interruptions matter here, and each one becomes a signal:

- **The user presses Ctrl+C.** The kernel sends `SIGINT`. The client should shut down cleanly: tell every server to quit, wait for them, then exit. A second Ctrl+C means “stop now”.
- **A server stops answering.** The client would wait in `read()` forever. An alarm clock (`SIGALRM`) interrupts the wait so the client can give up.
- **A server process dies.** The kernel tells its parent with `SIGCHLD`. The client collects (“reaps”) the dead child and marks the server as terminated in its registry.



**Figure 1.1:** The three signals in this lab and where each one comes from. Every server calls `setpgid(0, 0)` before `exec`, so Ctrl+C reaches *only* the client. The client then stops the servers itself, through the pipes.

### 1.3 What you edit, and how it is graded

You edit exactly two files: `signals.cpp` and `client.cpp`. The autograder takes *only* those two from your repository. It replaces the headers, the servers, the channel, the `Makefile` and `tests/` with its own copies, so changing any of those has no effect on your score.

| Autograder row                                            | Pts        | File                     | This guide |
|-----------------------------------------------------------|------------|--------------------------|------------|
| Atomic flags declared and initialised                     | 5          | <code>signals.cpp</code> | Task 1     |
| Handlers installed with <code>sigaction</code>            | 15         | <code>signals.cpp</code> | Task 2     |
| <code>SIGINT</code> handler                               | 10         | <code>signals.cpp</code> | Task 3     |
| <code>SIGALRM</code> handler                              | 10         | <code>signals.cpp</code> | Task 4     |
| <code>SIGCHLD</code> handler updates the server registry  | 10         | <code>signals.cpp</code> | Task 5     |
| <code>block_signals</code> blocks <code>SIGINT</code>     | 10         | <code>signals.cpp</code> | Task 6     |
| <code>unblock_signals</code> unblocks <code>SIGINT</code> | 10         | <code>signals.cpp</code> | Task 6     |
| <code>wait_with_timeout</code> arms the alarm             | 10         | <code>signals.cpp</code> | Task 7     |
| <code>cancel_timeout</code> clears a pending alarm        | 5          | <code>signals.cpp</code> | Task 7     |
| All three servers registered with the signal handlers     | 5          | <code>client.cpp</code>  | Task 8     |
| Signals blocked and unblocked around every transaction    | 10         | <code>client.cpp</code>  | Task 9     |
| <b>Total</b>                                              | <b>100</b> |                          |            |

Every `signals.cpp` row compiles a small test program from `tests/` against *your* `signals.cpp`

and runs it. The tests raise real signals and ask the kernel what happened. The two `client.cpp` rows run a complete client session.

#### You can read the tests

The test programs are in your repository: `tests/test_flags.cpp`, `test_setup.cpp`, `test_sigint.cpp`, and so on. Each is under 80 lines. When a row fails, open its test and read the failure message's `printf`. It names the exact check that failed.

The grader shows only the *last* message a test printed, so a row can have more problems than it reports. To see all of them, build and run that one test yourself from the repository directory:

```
g++ -std=c++11 -Wall -pthread -o test_sigint tests/test_sigint.cpp signals.cpp
./test_sigint # same pattern for test_flags, test_setup, test_sigalrm, ...
./test_blocking block # these two take an argument: block|unblock, arm|cancel
```

`make clean` removes the `test_*` binaries afterwards.

## 1.4 Build and test loop

Work in your VM, inside your Lab 4 repository:

```
cd ~/csce-313-fa26-lab-4-<your-username>
make # builds client, finance, logging, file
make test # runs the same checks as the autograder, on your code
make clean # removes binaries, FIFOs, logs, storage/
```

**The starter code does not build. This is expected.**

Before you do anything, `make` fails at the link step:

```
/usr/bin/ld: signals.o: in function 'SignalHandling::wait_with_timeout(int)':
signals.cpp:(.text+0x243): undefined reference to 'SignalHandling::
    timeout_occurred'
collect2: error: ld returned 1 exit status
make: *** [Makefile:15: finance] Error 1
```

and `make test` gives **0/100**, with every row saying `signals.cpp` does not define what `signals.h` declares. Nothing else can pass until **Task 1** is done, so do it first. It takes three lines.

#### Recommended order

1. Task 1 (flags). After this, `make` succeeds.
2. Tasks 2–7 in `signals.cpp`. Run `make test` after each one and watch its row go green.
3. Tasks 8–9 in `client.cpp`.
4. Chapter 5: press Ctrl+C yourself and check that the program really behaves correctly. The grader checks less than you might expect.

**make test takes about 20 seconds**

Several checks wait on real timers (one test waits for a 1-second alarm and then another 2 seconds to prove a cancelled alarm stays cancelled). The grader runs these one at a time.

Do not run two `make tests` at once in the same VM. The grader cleans up with `pkill -x client` (and the same for each server), so a second run kills the first run's processes and both report the session did not complete.

# Signals in Fifteen Minutes

Read this chapter once before starting the tasks. Every TODO in the lab is an application of one of the five ideas below.

## 2.1 Idea 1: a signal is a software interrupt

A **signal** is a small, numbered notification the kernel delivers to a process. It carries almost no data, just its number (`SIGINT` is 2, `SIGALRM` is 14, `SIGCHLD` is 17 on Linux). It can arrive at *any* instruction. Your program does not ask for it. It happens to your program.

Signals come from three places:

- **the terminal:** Ctrl+C makes the kernel send `SIGINT` to the foreground process group;
- **the kernel itself:** `alarm()` expires and you get `SIGALRM`; a child exits and its parent gets `SIGCHLD`;
- **a process:** `kill(pid, sig)` sends to another process, and `raise(sig)` sends to yourself. The tests use `raise()` a lot.

Every signal has a **disposition**, meaning what happens on arrival:

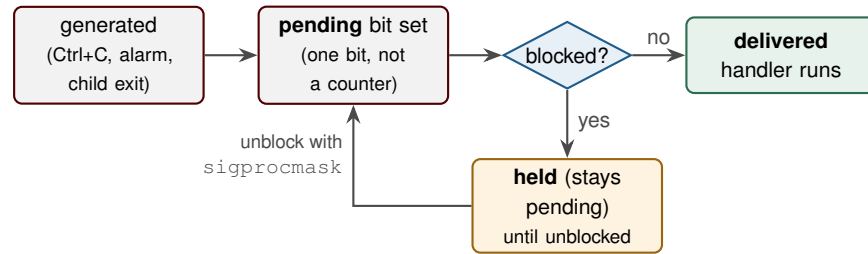
| Signal               | Default action        | What this lab does instead                  |
|----------------------|-----------------------|---------------------------------------------|
| <code>SIGINT</code>  | terminate the process | handler: set a flag (1st), force exit (2nd) |
| <code>SIGALRM</code> | terminate the process | handler: set the “timed out” flag           |
| <code>SIGCHLD</code> | ignore                | handler: reap children, update the registry |

Note the first two rows. With no handler installed, a timeout or a Ctrl+C simply *kills* your client and leaves three orphaned servers behind. Installing handlers is what turns “killed” into “handled”.

## 2.2 Idea 2: pending, blocked, delivered

A signal does not run its handler the moment it is generated. The kernel keeps two bit-sets per process: the **pending** set (generated but not yet delivered) and the **blocked mask** (signals the process has asked to hold).





**Figure 2.1:** The life of a signal. Blocking does not throw a signal away: it parks it in the pending set until you unblock.

Two consequences you will see in this lab:

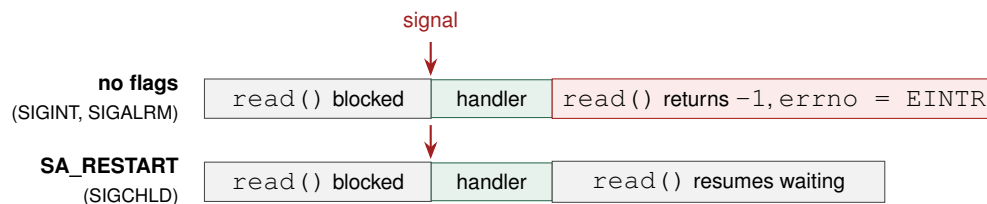
- **Blocked is not the same as ignored.** An ignored signal is discarded. A blocked signal waits. Task 6 depends on this: a Ctrl+C during a bank transaction is *postponed*, not lost.
- **Pending is one bit, so signals of the same type do not queue.** Five Ctrl+C presses while `SIGINT` is blocked leave *one* pending `SIGINT`. Three children exiting at nearly the same moment may produce only *one* `SIGCHLD`. Task 5's loop exists because of this.

#### The kernel blocks a signal while its own handler runs

While your `SIGINT` handler is running, the kernel automatically blocks `SIGINT`. A second Ctrl+C during the handler waits and is delivered right after the handler returns. Handlers therefore never interrupt themselves.

## 2.3 Idea 3: signals interrupt system calls

Suppose the client is sitting in `read()`, waiting for you to type a menu choice, and a signal arrives. The handler runs. When it returns, there are two possible behaviours, and **you** choose between them with the `SA_RESTART` flag:



**Figure 2.2:** The same signal with and without `SA_RESTART`.

- For `SIGINT` and `SIGALRM` you **want** the interruption. Ctrl+C must break the client out of `cin` » choice so it can see the shutdown flag. An alarm must break the client out of a `read()` on a dead server. That is the whole point of a timeout.
- For `SIGCHLD` you do **not** want it. A server exiting is bookkeeping. It should not make an unrelated `read()` or `write()` fail.

## 2.4 Idea 4: a handler may only call async-signal-safe functions

This is the most important rule in the lab, and the autograder **cannot** check it. A handler can interrupt *any* instruction, including one inside `malloc`, inside `std::cout`, or inside `localtime`. Suppose the interrupted code was halfway through `malloc` and holding its internal lock, and your handler builds a `std::string`, which calls `malloc`. The handler waits for a lock that the code it interrupted will never release. The process deadlocks, and it does so only sometimes, which makes it very hard to debug.

POSIX publishes a short list of **async-signal-safe** functions: the only ones you may call from a handler. For this lab:

| Safe in a handler                                                    | Not safe in a handler                                                                           |
|----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| <code>write</code> , <code>open</code> , <code>close</code>          | <code>printf</code> , <code>std::cout</code> , <code>std::cerr</code> , <code>perror</code>     |
| <code>_exit</code>                                                   | <code>exit</code> (runs atexit handlers, flushes stdio)                                         |
| <code>waitpid</code> , <code>alarm</code> , <code>sigprocmask</code> | <code>std::string</code> , <code>std::to_string</code> , <code>new</code> , <code>malloc</code> |
| reading/writing a lock-free <code>std::atomic</code>                 | <code>localtime</code> , <code>strftime</code> , <code>sprintf</code>                           |
| plain local arrays and loops                                         | <code>log_signal_event()</code> , which uses all of the above                                   |

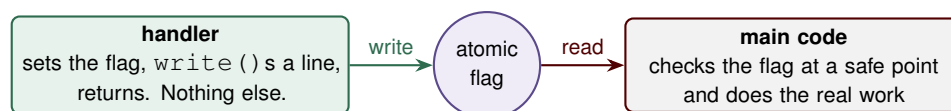
### log\_signal\_event() inside a handler passes every test and is still wrong

It is tempting to record each event by calling the provided `log_signal_event()` from your handler. The tests only check that *something* was written, so this passes **100/100**. It is still a real bug, and the TODO comments in `signals.cpp` forbid it explicitly. A TA reading your code will see it. Inside handlers, use `write()` only. Outside handlers (Task 6), `log_signal_event()` is fine.

## 2.5 Idea 5: share state with a handler through atomic flags

A handler takes no useful arguments and returns nothing. The only way it can tell the rest of the program “Ctrl+C happened” is to write a variable that the main code later reads. That variable must be safe to write from a handler and must not be cached in a register by the compiler. In C you would use `volatile sig_atomic_t`. This lab uses C++’s `std::atomic<bool>` and `std::atomic<int>`, which provide the same guarantee and also work across threads.

The pattern used everywhere in this lab:



**Figure 2.3:** Handlers set flags. Normal code acts on them. The client’s menu loop checks `shutdown_requested` on every iteration and does the complicated shutdown (sending QUIT, waiting for children) outside any handler.

**The five ideas**

1. Signals arrive asynchronously. Without a handler, `SIGINT` and `SIGALRM` kill the process.
2. Blocking holds a signal pending. It is not lost, and it does not queue.
3. Without `SA_RESTART` a blocked system call fails with `EINTR`; with it, the call resumes.
4. Handlers call only async-signal-safe functions: `write`, `_exit`, `waitpid`, and `atomic`s.
5. Handlers set flags. Normal code does the work.

## Part A — signals.cpp

Seven tasks, 85 points. Every one is checked by a small test program that is compiled against your `signals.cpp` alone, so you can finish and verify this whole chapter before touching `client.cpp`.

### The “Declare functions” TODOs need no code

Two TODOs read `// TODO: Declare functions for signal handlers ...` and `// TODO: Declare functions for signal operations ....` Those functions are already declared in `signals.h`, and their bodies already exist below each comment. These two comments are only section labels. The work is in the TODOs *inside* each function.

### 3.1 Task 1 — Define the three atomic flags

#### Commands used in this task

|                                      |                                                                |
|--------------------------------------|----------------------------------------------------------------|
| <code>std::atomic&lt;bool&gt;</code> | a bool that is safe to write from a handler and read elsewhere |
| <code>std::atomic&lt;int&gt;</code>  | the same, for a counter; supports ++ atomically                |
| <code>extern</code>                  | “this variable exists, somewhere else” — a declaration only    |

#### What this task asks you to do

At line 15 of `signals.cpp`, define the three flags that `signals.h` declares: `shutdown_requested` (bool), `timeout_occurred` (bool) and `child_exited` (int). `test_flags` checks that they exist, that they are really `std::atomic`, and that they start as false, false and 0. **5 points**, but every other row depends on it: until this is done, nothing links.

#### How to solve it

Open `signals.h` and look at lines 13–15. Each flag is declared with `extern`. That line is a *promise* that the variable is defined in exactly one `.cpp` file somewhere. Your job is to keep that promise.

The general pattern, using a made-up variable so you can see the shape:

```
// counter.h -- declaration (no storage, can be included everywhere)
namespace Stats { extern int hits; }

// counter.cpp -- definition (storage lives here, exactly once)
namespace Stats { int hits(0); }
```

Apply that to the three flags:

1. Same **type** as the `extern` line (copy it, then remove `extern`).
2. Same **namespace**. Line 15 is already inside `namespace SignalHandling { ... }`, so write them there.
3. Give each an explicit **initial value** using parentheses or braces. See the red box below for why `= false` fails.

#### “use of deleted function”: `atomic` and `the = sign`

This lab builds with `-std=c++11`. In C++11 an `atomic` cannot be *copy*-initialised, so the natural-looking `std::atomic<bool> x = false;` does not compile:

```
error: use of deleted function 'std::atomic<bool>::atomic(const std::atomic<bool>&)'
```

Use direct initialisation instead: `x(false)` or `x{false}`. (C++17 accepts the `=` form, which is why code you find online may use it.)

#### Three ways to still get “undefined reference”

- Writing `extern` again in the `.cpp`. That is another declaration, not a definition.
- Defining them *outside* `namespace SignalHandling`. That creates `::shutdown_requested`, a different variable from `SignalHandling::shutdown_requested`.
- Defining them in `signals.h` instead. Every `.cpp` that includes the header gets its own copy, and the link fails with `multiple definition`. The grader also replaces your `signals.h`, so the definitions would disappear anyway.

#### Declaration vs. definition: what the linker is telling you

The compiler handles one `.cpp` at a time. When it compiles `channel.cpp` and sees `SignalHandling::timeout_occurred = false;`, the `extern` declaration is enough. It emits a note saying “some other object file has this symbol” and moves on. The **linker** later joins all the `.o` files and has to resolve every such note. The starter’s `signals.cpp` defines none of the three, so every note is unresolved: `undefined reference to 'SignalHandling::timeout_occurred'`.

That is why one missing three-line block zeroes all eleven rows. Every test, the client and all three servers link against `signals.o`.

#### Where you will use this for real

- “undefined reference” and “multiple definition” are the two most common linker errors in any C or C++ codebase. You now know exactly what each one means.
- Global configuration such as `errno`, `environ`, or a library’s verbosity level is declared `extern` in a public header and defined once inside the library.
- Graceful-shutdown flags like `shutdown_requested` appear in servers everywhere. `nginx`, `PostgreSQL` and `Redis` all set a flag in a `SIGTERM/SIGINT` handler and check it in their main loop.

**The intuition**

A header is a phone book: it says a person exists and how to reach them. A definition is the person. You can print the phone book in every file, but the person must live in exactly one place. If nobody lives there, every call fails (“undefined reference”). If two people claim the same entry, the linker refuses to choose (“multiple definition”).

**Takeaway**

Define `shutdown_requested`, `timeout_occurred` (both `std::atomic<bool>`, initially `false`) and `child_exited` (`std::atomic<int>`, initially `0`) inside namespace `SignalHandling` in `signals.cpp`, using `(...)` or `{...}` initialisation because this is C++11. After this, `make` succeeds and the first row reads `atomic flags defined and initialised correctly`.

**Checkpoint**

`make` finishes with no errors and produces `client`, `finance`, `logging` and `file`. `make test` now shows real per-row results instead of eleven link errors.

## 3.2 Task 2 — Install the three handlers with sigaction

**Commands used in this task**

|                                                 |                                                                 |
|-------------------------------------------------|-----------------------------------------------------------------|
| <code>struct sigaction</code>                   | describes a disposition: handler, mask, flags                   |
| <code>sa.sa_handler</code>                      | the function to run: <code>void f(int)</code>                   |
| <code>sa.sa_flags</code>                        | behaviour bits, here <code>0</code> or <code>SA_RESTART</code>  |
| <code>sigaction(sig, &amp;new, &amp;old)</code> | install new for <code>sig</code> ; old may be <code>NULL</code> |

**What this task asks you to do**

Complete `setup_handlers()` (lines 21–33). Install `sigint_handler` for `SIGINT` and `sigalrm_handler` for `SIGALRM` with **no flags**, and `sigchld_handler` for `SIGCHLD` with **`SA_RESTART`**. `test_setup` asks the kernel what is installed for each signal and checks both the handler and the flags. **15 points**, all or nothing.

**How to solve it**

The function already declares `sa`, zeroes it, and empties its mask. What is left is three installations. The shape of one, shown for a signal this lab does *not* use:

```
sa.sa_handler = on_usr1; // which function
sa.sa_flags = 0; // which behaviour bits
if (sigaction(SIGUSR1, &sa, NULL) == -1)
    perror("sigaction SIGUSR1"); // fine here: we are NOT in a handler
```

Do this three times, using the table:

| Signal  | Handler         | Flags      | Why (Sec. 2.3)                 |
|---------|-----------------|------------|--------------------------------|
| SIGINT  | sigint_handler  | 0          | must break <code>cin</code>    |
| SIGALRM | sigalrm_handler | 0          | must break <code>read()</code> |
| SIGCHLD | sigchld_handler | SA_RESTART | must not break anything        |

Set `sa_flags` explicitly before every call. The same `sa` struct is reused, so whatever you set for one signal carries over to the next unless you overwrite it.

#### signal() looks like it works and scores 0/15

`signal(SIGINT, sigint_handler)` does install the handler, but on Linux (glibc) it *also* turns on `SA_RESTART`. The grader reports:

```
Handlers installed with sigaction 0/15 - SIGINT was installed with
SA_RESTART; the task asks for default flags on this one
```

It is a real behaviour bug, not a technicality. With `SA_RESTART` on `SIGINT`, pressing `Ctrl+C` at the menu **does nothing visible**. The client keeps waiting in `cin` and only shuts down after you press `Enter`. We tested this. Use `sigaction`.

#### Flags leaking from one signal to the next

If you install `SIGCHLD` first with `SA_RESTART` and then install the other two without resetting `sa_flags`, they inherit it: `SIGALRM` was installed with `SA_RESTART`; the task asks for default flags on this one. The reverse mistake gives `SIGCHLD` is installed but without `SA_RESTART`.

#### What sigaction actually stores

For each signal number the kernel keeps one disposition record: a handler pointer, a mask of extra signals to block while that handler runs, and a flags word. `sigaction()` replaces that record. Passing a non-NULL third argument reads back the previous one. `test_setup` uses exactly that query form, `sigaction(SIGINT, NULL, &current)`, which changes nothing and returns what is installed. That is how it can tell `SA_RESTART` from no flags, and why `signal()` is caught.

The handler must be *these* functions. The test compares the stored pointer with `SignalHandling::sigint_handler` and the others, so a different function that does the same thing still fails.

#### Where you will use this for real

- Every long-running server installs handlers at startup. `nginx` maps `SIGHUP` to “reload config”, `SIGUSR1` to “reopen log files”, and `SIGTERM/SIGINT` to shutdown. Each one is a `sigaction` call.
- The `SA_RESTART` decision appears in real code reviews. Network daemons often leave it off so a signal can break them out of `accept()` or `poll()`.

- `signal()`'s behaviour differs between Unix variants (System V resets the handler after one delivery; BSD and glibc restart system calls). That inconsistency is why POSIX recommends `sigaction`.

#### The intuition

`sigaction` fills in a form the kernel keeps for each signal: “when this arrives, call this function, and afterwards, retry or don’t retry whatever I was waiting on”. `signal()` fills in the same form but ticks the retry box for you. In this lab that box has to be left blank for Ctrl+C.

#### Takeaway

Three `sigaction` calls: `SIGINT` and `SIGALRM` with `sa_flags = 0`, `SIGCHLD` with `sa_flags = SA_RESTART`, each with its own `SignalHandling::*_handler`. Do not use `signal()`. The passing row reads all three handlers installed, `SA_RESTART` on `SIGCHLD`.

### 3.3 Task 3 — The SIGINT handler: flag first, forced exit second

#### Commands used in this task

|                                      |                                                                                       |
|--------------------------------------|---------------------------------------------------------------------------------------|
| <code>write(fd, buf, n)</code>       | raw, unbuffered output; async-signal-safe                                             |
| <code>STDOUT_FILENO</code>           | file descriptor 1 (also <code>STDERR_FILENO = 2</code> )                              |
| <code>open(path, flags, 0644)</code> | open a file; for a log use <code>O_WRONLY O_CREAT O_APPEND</code> ; safe in a handler |
| <code>_exit(status)</code>           | terminate immediately, skipping all cleanup                                           |

#### What this task asks you to do

Fill in the two branches already present in `sigint_handler` (lines 37–54). On the **first** `SIGINT`: set `shutdown_requested` and record the event. On the **second**: record it and terminate the process with `_exit`. `test_sigint` raises one `SIGINT` and checks the flag and that *something* was written to `stdout`, `stderr` or `signals.log`. It then raises two in a child process and checks that the child died. **10 points**.

#### How to solve it

**Recording an event with `write()` only.** `write` needs a byte count. For a fixed message, let the compiler count:

```
static const char msg[] = "Something happened\n";
write(STDOUT_FILENO, msg, sizeof(msg) - 1); // -1: don't write the '\0'
```

Any of the three destinations passes the test. `stdout` is the easiest to see while testing, because the message appears right where you pressed Ctrl+C. Appending to `signals.log` (open, write, close, all safe) keeps your log complete. Some students do both. Since you will need the same thing in three handlers, a small helper of your own is a good idea, as long as it only uses safe calls.



**The logic, as pseudo-code:**

```

if shutdown has NOT been requested:
    mark shutdown as requested
    write "SIGINT received - initiating graceful shutdown"
else:
    write "Second SIGINT received - forcing exit"
    _exit(<any non-zero status>) // 130 = 128 + SIGINT, the shell convention

```

**exit() vs \_exit()**

`exit()` runs `atexit` handlers, global destructors and stdio flushing. None of those are async-signal-safe, and any of them may be what the program was in the middle of when Ctrl+C arrived. `_exit()` goes straight to the kernel. The TODO says `_exit` for this reason.

**The grader cannot catch a missing `_exit`, so check it yourself**

We tested this: with the `_exit` call deleted, the row still scores 10/10. (Your own `SIGCHLD` handler from Task 5 collects the test's child before the test can see its exit status.) A TA reading your code will still see the bug. Check it with this throw-away program, which is a test and not part of your submission:

```

// probe.cpp -- g++ -std=c++11 -pthread probe.cpp signals.cpp -o probe &&
./probe; echo $?
#include "signals.h"
#include <stdio>
int main() {
    SignalHandling::setup_handlers();
    raise(SIGINT); printf("after 1st (expected)\n"); fflush(stdout);
    raise(SIGINT); printf("after 2nd (BUG if you see this)\n");
    return 42;
}

```

Correct: one “after 1st” line, no “after 2nd” line, and an exit status that is not 42. Delete `probe.cpp` and `probe` afterwards.

**How one flag shuts down four processes**

Follow a Ctrl+C at the menu, step by step:

1. The terminal sends `SIGINT` to its foreground process group. The servers called `setpgid(0,0)` before `exec`, so they are in their own groups. Only the client receives it.
2. The client is blocked in `read()` under `cin » choice`. Your handler runs and sets the flag. Because `SIGINT` has no `SA_RESTART`, the `read()` fails with `EINTR`.
3. `cin` enters the fail state, so `if (!(cin » choice))` is true. The very next line checks `shutdown_requested` and breaks out of the menu loop.
4. Outside any handler, `main` does the real work: prints `Performing graceful shutdown...`, sends `QUIT` to each server through its pipe, and `wait()`s for all three.

The handler did one thing, set a flag, and everything difficult happened in normal code where it is safe. The second Ctrl+C is the escape hatch for when that normal code is itself stuck, for example on a server that never answers.

**Where you will use this for real**

- “Press Ctrl+C again to force quit” is common: `docker compose up`, `npm run dev`, Python’s `asyncio`, many database shells. This is the pattern behind it.
- Kubernetes stops a container with `SIGTERM`, waits a grace period, then sends `SIGKILL`. That is the same two-stage idea run by an orchestrator.
- Exit status 130 is how shells and CI systems know a job was interrupted by Ctrl+C rather than failing on its own.

**The intuition**

The first Ctrl+C is a polite request: “please finish up”. The program keeps control and cleans up properly. The second is an order: “stop now”. You give up the cleanup in exchange for certainty. A good program honours both, and the handler is only a messenger in each case.

**Takeaway**

First `SIGINT`: `shutdown_requested = true`, plus a `write()`. Second `SIGINT`: a `write()`, then `_exit(non-zero)`. Never call `log_signal_event`, `cout` or `exit` here. Graceful shutdown works only because `SIGINT` has no `SA_RESTART` (Task 2). The passing row reads `SIGINT: flag set, event recorded, second press exits`.

### 3.4 Task 4 — The SIGALRM handler

**Commands used in this task**

|                               |                                                                           |
|-------------------------------|---------------------------------------------------------------------------|
| <code>timeout_occurred</code> | the flag <code>send_request()</code> checks after its <code>read()</code> |
| <code>write()</code>          | the same recording approach as Task 3                                     |

**What this task asks you to do**

Complete `sigalrm_handler` (lines 56–62): set `timeout_occurred` and record the event, suggested text “SIGALRM received - operation timed out”. `test_sigalrm` raises `SIGALRM` and checks both. **10 points.**

**How to solve it**

This is the first branch of Task 3 with a different flag and message: set the flag, `write()` one line, return. Do not cancel or re-arm the alarm here, and do not exit. Deciding what a timeout *means* is the caller’s job.

**Setting the flag is not enough**

If the handler only sets the flag, the row fails with the `SIGALRM` handler recorded nothing - it must report the timeout (`write()` to the log or to standard output). Both halves are graded.

### How a timeout actually ends a blocked read

Look at `RequestChannel::send_request` in `channel.cpp`. It calls `wait_with_timeout()` (Task 7) to arm an alarm, writes the request, then blocks in `read()` waiting for the reply. If the server is hung, that `read()` would never return. When the alarm expires:

1. the kernel delivers `SIGALRM` and your handler sets `timeout_occurred`;
2. with no `SA_RESTART`, the blocked `read()` returns `-1` with `errno = EINTR`;
3. `send_request` sees the failure and returns a failed `Response`, and the menu offers `Retry? (y/n)`.

The alarm and the flag together turn “wait forever” into “wait at most  $n$  seconds, then report”.

### What a real timeout looks like

We froze the logging server (`kill -STOP <pid>`) and then tried to log in. This is the actual output:

```
Enter user ID: 1
SIGALRM received - operation timed out (your handler's write)
Read failed: Interrupted system call (perror in send_request)
Login timed out after 10 seconds.
Operation failed. Retry? (y/n):
```

“Interrupted system call” is how `EINTR` reads in plain English. It is the timeout working as designed, not a separate bug. The message says 10 seconds but the wait was actually **30**. Task 7 explains why. `kill -CONT <pid>` un-freezes the server.

### Where you will use this for real

- `curl -max-time`, `ssh`’s `ConnectTimeout`, and the `timeout` command (`timeout 5 ./prog`) all bound how long something may block. `timeout` literally sends a signal when time runs out.
- Watchdog timers in embedded systems and kernels: if the main loop does not check in within  $n$  ms, a timer interrupt fires and recovers the system.
- Every RPC framework (gRPC deadlines, HTTP client timeouts) exists because “the other side might never answer” is a normal failure in distributed systems.

### The intuition

A blocking `read()` on a pipe has no sense of time. The alarm adds one: an egg timer the kernel holds for you. When it rings, it does not fix anything. It interrupts the wait and leaves a note (the flag) saying why. The code that was waiting reads the note and decides what to do.

### Takeaway

`sigalrm_handler: timeout_occurred = true` plus one `write()`. Nothing else. The timeout works because `SIGALRM` has no `SA_RESTART`, so the pending `read()` fails with `EINTR`. The passing row reads `SIGALRM: timeout flag set and event recorded`.

### 3.5 Task 5 — The SIGCHLD handler: reap, mark inactive, record

#### Commands used in this task

|                                                |                                                                       |
|------------------------------------------------|-----------------------------------------------------------------------|
| <code>waitpid(-1, &amp;status, WNOHANG)</code> | reap <i>any</i> dead child; return 0 instead of blocking if none      |
| <code>server_processes</code>                  | vector of {pid, name, active}, filled by <code>register_server</code> |
| <code>child_exited</code>                      | atomic counter of children reaped                                     |

#### What this task asks you to do

Inside the existing for loop in `sigchld_handler` (lines 64–81), find the registry entry whose pid matches the child just reaped, mark it inactive, and record "Child process terminated, PID: <pid>" using `write()` only. The `waitpid` loop and the `child_exited++` are already written. `test_sigchld` forks a child, registers it as "probe", lets it exit, and checks four things: the entry is inactive, the counter grew, an event was recorded, and the child was actually reaped. **10 points.**

#### How to solve it

1. Inside the `for (auto& server : server_processes)` loop: if this entry's pid equals the reaped pid, set its active to false. (auto& is a reference, so you are changing the real entry, not a copy.) Once you have found it, there is no reason to keep looking.
2. Build the message. The hard part is putting the PID's digits into text *without* `printf`, `to_string` or `stringstream`. Do it by hand into a local char array:

```
// turn a positive number n into decimal text, e.g. 3527 -> "3527"
tmp = empty char array, k = 0
do: tmp[k++] = '0' + (n % 10); n = n / 10; while n > 0
// tmp now holds the digits BACKWARDS ("7253"): copy them out in reverse,
// after the fixed prefix, into your output buffer; track the length as
// you go
```

3. `write()` the finished buffer with the length you tracked.

#### Save and restore errno

`waitpid` sets `errno` (to `ECHILD` when there is nothing left to reap). If the code you interrupted was about to check `errno` for its own reasons, the handler has just changed it underneath that code. Careful handlers save it on entry and restore it on exit: `int saved = errno; ... errno = saved;`. This is not graded, but it is good practice in all three handlers.

#### Zombies, reaping, and why it is a while loop

When a child exits, the kernel cannot discard it yet, because its parent may want the exit status. The child becomes a **zombie**: no memory, no code, just a process-table entry holding its status. The parent **reaps** it with `wait/waitpid`, which returns the status and frees the entry. A parent that never reaps slowly fills the process table.

`SIGCHLD` is the kernel saying “one of your children changed state”. It is one pending bit (Sec. 2.2), so if two servers exit before the handler runs, you get *one* signal for *two* zombies. That is why the given code is a `while` loop over `waitpid(-1, ..., WNOHANG)`: keep reaping until nothing is left, and never block (`WNOHANG`), because a handler must not wait.

`SIGCHLD` gets `SA_RESTART` because a server exiting has nothing to do with whatever the client happens to be reading or writing at that moment. That operation should continue as if nothing happened.

#### “But it is iterating a `std::vector` inside a handler”

Reading and changing fields of *existing* elements is fine: no allocation, no locks. What would be unsafe is `push_back` or anything else that can reallocate the vector. The handler never does that. `register_server` (in normal code) does, so registering always happens outside the handler.

#### Where you will use this for real

- Your shell does exactly this. When a background job (`sleep 5 &`) finishes, `bash`’s `SIGCHLD` handler reaps it and later prints `[1]+ Done`.
- Process supervisors (`systemd`, `supervisord`, Docker’s `tini`) are mostly a `SIGCHLD` handler plus a registry like `server_processes`, with a restart policy added.
- Zombie buildup in containers is a classic production bug: if PID 1 does not reap orphans, `ps` fills with `<defunct>` entries. `tini` exists to fix that.

#### The intuition

A zombie is a sealed envelope addressed to the parent containing “how I died”. `SIGCHLD` is the doorbell. The doorbell rings once even if three envelopes arrived, so you keep collecting until the mailbox is empty (`while` plus `WNOHANG`), and you write each sender down in your registry.

#### Takeaway

Inside the existing loop, find the registry entry whose PID matches the child just reaped, mark it inactive, and `write()` a `Child process terminated, PID: <digits>` line, building the digits by hand (remainder and division by ten, then reverse). No `printf`, `to_string` or `log_signal_event`. The passing row reads `SIGCHLD: child reaped, registry updated, event recorded`.

## 3.6 Task 6 — `block_signals()` and `unblock_signals()`

**Commands used in this task**

|                                               |                                                                                                               |
|-----------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| <code>sigset_t set</code>                     | a bit-set of signal numbers                                                                                   |
| <code>sigemptyset(&amp;set)</code>            | clear every bit (always do this first)                                                                        |
| <code>sigaddset(&amp;set, SIGINT)</code>      | set one bit                                                                                                   |
| <code>sigprocmask(how, &amp;set, NULL)</code> | change the blocked mask; how is <code>SIG_BLOCK</code> , <code>SIG_UNBLOCK</code> or <code>SIG_SETMASK</code> |

**What this task asks you to do**

Implement `block_signals()` (lines 85–93) to add `SIGINT` to the process's blocked mask, and `unblock_signals()` (lines 95–103) to remove it. Each must then call `log_signal_event()` with *exactly* "Signals blocked for critical section" or "Signals unblocked". `test_blocking` reads the real mask from the kernel and also checks that a `SIGINT` raised while blocked is *held*, then delivered on unblock. **10 + 10 points.**

**How to solve it**

Each function is three steps plus the log line:

```
build an empty signal set
add SIGINT to it
sigprocmask( <SIG_BLOCK or SIG_UNBLOCK>, &set, NULL )
log_signal_event( <exact message> ) // safe: this is normal code, not a handler
```

**SIG\_BLOCK / SIG\_UNBLOCK, not SIG\_SETMASK**

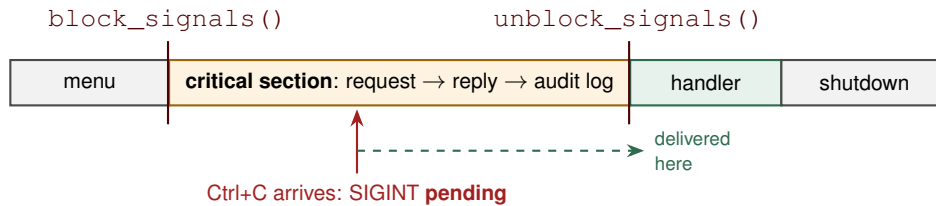
`SIG_BLOCK` *adds* your set to the current mask and `SIG_UNBLOCK` *removes* it. `SIG_SETMASK` *replaces* the whole mask, which silently unblocks anything else that was blocked. Common failure messages:

- after `block_signals()`, `SIGINT` is not in the process mask; use `sigemptyset`, `sigaddset(SIGINT)` and `sigprocmask(SIG_BLOCK, ...)`. This usually means the set was built but `sigprocmask` was never called, or was called with the wrong set.
- after `unblock_signals()`, `SIGINT` is still blocked; use `sigprocmask(SIG_UNBLOCK, ...)`. Usually a copy-paste that kept `SIG_BLOCK`.

If `block_signals` fails, the `unblock` row fails too (`block_signals()` did not block `SIGINT`, so unblocking cannot be checked). Fix `block` first.

**The two log messages are exact strings**

The TODOs say “strictly use the following message”. Copy them character for character: Signals blocked for critical section and Signals unblocked.



**Figure 3.1:** A Ctrl+C in the middle of a deposit is held until the deposit finishes, and its handler runs inside the `sigprocmask` call that unblocks it.

#### Why “held” and not “ignored” is the point

The test does something precise. It blocks, `raise(SIGINT)`s, checks that the handler has *not* run, then unblocks and checks that it *has*. That is the definition of a correct critical section. The interrupt is **deferred**, not lost. When `sigprocmask(SIG_UNBLOCK)` finds a pending signal, the kernel delivers it before `sigprocmask` even returns, so the handler runs at that exact point in your code.

Why bother? A deposit is two messages: one to `finance` (move the money) and one to `logging` (record that it moved). If the user could interrupt between them, the bank would hold money with no audit record. Blocking `SIGINT` for that window makes the pair all-or-nothing from Ctrl+C’s point of view.

#### Where you will use this for real

- Databases commit in exactly this shape: begin, apply changes, write the log record, commit. Interrupting halfway is what transactions and write-ahead logs exist to prevent.
- The Linux kernel briefly disables interrupts around code that must not be split (`local_irq_save/restore`). This is the same idea one level down.
- Bash blocks `SIGCHLD` while it updates its job table, so its own `SIGCHLD` handler cannot see the table half-edited.
- Installers and package managers (`apt`, `dpkg`) defer Ctrl+C while writing their state files, so an interrupted install does not corrupt the package database.

#### The intuition

Blocking a signal is putting your phone on Do Not Disturb, not throwing it in a lake. Calls during the meeting are still waiting when you come out, but you only see *that* someone called, not how many times. Use it for short stretches where an interruption would leave things half-done, and always turn it off again.

#### Takeaway

`block_signals`: `sigemptyset`, `sigaddset(SIGINT)`, `sigprocmask(SIG_BLOCK)`, then `log_signal_event("Signals blocked for critical section")`. `unblock_signals`: the same with `SIG_UNBLOCK` and `"Signals unblocked"`. A blocked `SIGINT` stays pending and is delivered inside the unblocking call.

### 3.7 Task 7 — `wait_with_timeout()` and `cancel_timeout()`

#### Commands used in this task

|                       |                                                                                                    |
|-----------------------|----------------------------------------------------------------------------------------------------|
| <code>alarm(n)</code> | deliver <code>SIGALRM</code> in <code>n</code> seconds; returns seconds left on the previous alarm |
| <code>alarm(0)</code> | cancel any pending alarm                                                                           |

#### What this task asks you to do

`wait_with_timeout(seconds)` (lines 105–112) must clear `timeout_occurred` *and then* arm an alarm for `seconds`, and return `true`. `cancel_timeout()` (lines 114–117) must cancel any pending alarm. `test_timeout` checks that the flag is cleared, that an alarm of at most 5 s is pending after `wait_with_timeout(5)`, that a 1 s alarm actually fires, and that a cancelled alarm stays silent. **10 + 5 points.**

#### How to solve it

Both are one or two lines. The flag reset is already in the starter. Keep it *above* the line you add. `cancel_timeout` is a single call.

#### Order matters: clear the flag, then arm

If the flag is not cleared, the row fails with `wait_with_timeout()` left `timeout_occurred` set; it must clear the flag before arming the alarm, or the very next operation looks like it timed out. Clear first, then arm. If you armed first, a very short alarm could fire between the two lines, and your reset would erase a real timeout.

#### One alarm per process

A process has **one** alarm clock, not a queue of them. `alarm(n)` replaces whatever was set and returns the seconds that were left on the old one; `alarm(0)` cancels. The test uses that return value as a probe: it calls `alarm(0)` right after your `wait_with_timeout(5)` and expects a number between 1 and 5.

“Replaces” has a visible effect in this program. Each menu option wraps its request in `execute_with_timeout(..., 10)`, which calls `alarm(10)`. `send_request()` then calls `wait_with_timeout(30)` with its default argument, which calls `alarm(30)` and *replaces* the 10. So the “Login timed out after 10 seconds” message actually appears after about 30 seconds. We measured it: 30.1 s with a frozen server. This is in the provided code, not yours, and it is a good example of why shared global state like the one alarm clock is easy to misuse.

#### The servers call `alarm(0)` too, harmlessly

The servers call `receive_request(0)`, so they run `wait_with_timeout(0)`, which is `alarm(0)`: no alarm at all. That is intended. A server should wait for a client as long as it takes.



**Where you will use this for real**

- `alarm()` is the oldest Unix timeout. Modern code uses `setitimer`, `timer_create`, `timerfd`, or a `timeout` argument to `poll/epoll_wait`, all because one process-wide alarm does not scale to many concurrent operations.
- “Arm a timer, do the risky thing, disarm the timer” is the shape of every deadline: `context.WithTimeout` in Go, `asyncio.wait_for` in Python, request timeouts in web servers.
- Forgetting to disarm is a classic bug: the timer fires later, during some unrelated operation, and that operation mysteriously “times out”. `cancel_timeout()` is the disarm.

**The intuition**

There is one kitchen timer on the counter. Setting it for 5 minutes wipes out the 10 that someone else set. Before you start it, clear the “it rang” note from last time. When you are done early, switch it off, or it will ring during the next person’s cooking.

**Takeaway**

`wait_with_timeout` clears the timeout flag *first*, then arms the alarm for the requested number of seconds, and reports success. `cancel_timeout` asks for an alarm of zero seconds, which cancels the pending one. There is one alarm per process, and a new one replaces the old one. Passing rows: `wait_with_timeout()`: flag reset, alarm armed, alarm fires and `cancel_timeout()`: pending alarm cleared.

**End of Part A**

`make test` should now show all nine `signals.cpp` rows passing (85/100). The two `client.cpp` rows still fail, because the servers are not registered yet:

```
lab-4 local tests: 85/100
All three servers registered ... 0/5 - no server was registered:
  call register_server(pid, name) in the parent after each fork,
  with the names finance, logging, file
Signals blocked and unblocked ... 0/10 - client.cpp calls block_signals
  0 time(s) and unblock_signals 0 time(s); each of the 7 transactions
  needs one of each, around the call that talks to the server
```

That is Part B.

## Part B — client.cpp

Two tasks, 15 points. Both are small edits at TODOs that are already in the right places. The skill here is understanding *which process* runs each line and *which code* needs protecting.

### How the grader judges client.cpp

On the real autograder, your `client.cpp` is linked against a *reference* `signals.o`, so these two rows depend only on `client.cpp`. A broken `signals.cpp` cannot zero them there. Locally, `make test` uses your own `signals.cpp` for everything, so finish Part A first or these rows may fail for reasons that have nothing to do with `client.cpp`.

### 4.1 Task 8 — Register the three servers

#### Commands used in this task

|                                         |                                                          |
|-----------------------------------------|----------------------------------------------------------|
| <code>fork()</code>                     | returns the child's PID to the parent, 0 to the child    |
| <code>register_server(pid, name)</code> | add {pid, name, active=true} to server_processes         |
| menu option 8                           | <code>print_server_status()</code> : prints the registry |

#### What this task asks you to do

At the three `// TODO: Register ... server` lines (143, 178, 233), call `SignalHandling::register_server` with the PID of the server just forked and its name: exactly `finance`, `logging` and `file`. The grader runs a full session and checks that `signals.log` contains `Registered server: <name> (PID: <n>)` for all three, and that menu option 8 shows all three `ACTIVE`. **5 points.**

#### How to solve it

For each of the three servers:

1. Find the `fork()` just above the `TODO` and the variable that holds its return value. Read carefully: the first two forks store it in `pid`, the third in a *different* variable.
2. Make sure the `TODO` line runs only in the **parent**. It already does: the child's `if (... == 0)` block always ends in `execvp` or `exit(1)`, so the child never reaches it. Write the call on the `TODO` line, not inside the child block.
3. Pass that PID and the server's name as a string literal.

**Registering inside the child block does nothing**

If you put the call inside `if (pid == 0) { ... }`, it runs in the *child*, which adds the entry to the child's own copy of `server_processes`. That is the memory isolation you measured in Lab 2. The child then `execs` and the copy disappears. The parent's registry stays empty: no server was registered: call `register_server(pid, name)` in the parent after each `fork`, with the names `finance`, `logging`, `file`.

**The file server's PID is in `file_pid`, not `pid`. The grader will not tell you.**

At the third TODO, `pid` still holds the **logging** server's PID. We tested `register_server(pid, "file")`: it scores 5/5, because option 8 still prints three `ACTIVE` lines. But the registry now says `file` has logging's PID. The `SIGCHLD` handler can never match the real file server's PID, so the `file` entry follows the wrong process. Compare the PIDs that option 8 prints with `pgrep -l -x file` in a second terminal.

**Names are exact**

"Finance" or "file\_server" fails: only `file`, `logging` registered; all three servers must be. The grader matches the lowercase names exactly.

**A registry turns a PID into a meaning**

The `SIGCHLD` handler learns only one thing from the kernel: "process 3527 exited". A number is useless to a person. `register_server` writes down, at the moment you still know it, that 3527 is the `file` server. Later the handler can turn the number back into a name and option 8 can say `file (PID: 3527): TERMINATED`.

The only place that knows the mapping is the parent, right after `fork()` returns the child's PID. That is why the registration has to happen there and nowhere else.

**Where you will use this for real**

- `systemd`, `supervisord` and container runtimes all keep this exact table (PID → service name, state, restart count). `systemctl status` is option 8 with more columns.
- PID files (`/run/nginx.pid`) are a registry written to disk, so that other programs can find and signal a daemon later.
- Shell job control (`jobs`, `fg %1`) is a registry of your background children, updated by the shell's `SIGCHLD` handler.

**The intuition**

`fork()` is a birth, and the return value is the birth certificate number. The parent has to file it right away, because it is the only one who saw the birth. Later, when the kernel reports "certificate 3527 has died", the registry is what tells you who that was.

**Takeaway**

Three calls, in the parent, after each `fork`: `register_server(<that fork's PID variable>, "finance" / "logging" / "file")`. The third uses `file_pid`. Check with option 8, which should show all three `ACTIVE` with three *different* PIDs.

**What option 8 should print**

```

=== Server Status ===
finance (PID: 3525): ACTIVE
logging (PID: 3526): ACTIVE
file (PID: 3527): ACTIVE
=====

```

The PIDs will differ on your machine. They should be three consecutive or nearly consecutive numbers, all different.

**4.2 Task 9 — Protect every transaction with block/unblock****Commands used in this task**

|                                   |                                                           |
|-----------------------------------|-----------------------------------------------------------|
| <code>block_signals()</code>      | your Task 6 function: hold <code>SIGINT</code>            |
| <code>unblock_signals()</code>    | release it; any held <code>SIGINT</code> is delivered now |
| <code>retry_operation(...)</code> | runs one transaction, with up to 3 attempts               |

**What this task asks you to do**

Menu options 1–7 each run one **transaction** through `retry_operation(...)`. Each has a pair of TODOs around that call: “*Implement signal blocking before transaction*” and “*... unblocking after transaction*”. Fill all seven pairs. The grader counts at least 7 `block_signals()` and 7 `unblock_signals()` calls in your `client.cpp` (comments are stripped first) *and* requires a full session to still complete. **10 points.**

**How to solve it**

The seven pairs are at lines 313/317 (login), 364/368 (deposit), 417/421 (withdraw), 464/468 (balance), 524/528 (upload), 582/586 (download) and 623/627 (logout). Each case has the same shape:

```

case N: {
    if (not allowed) { print; break; } // early exits: BEFORE blocking
    read the user's input (amount, filename...)
    auto op = [&]() { ... talk to the server ... };

    // TODO: blocking -> block here
    retry_operation("...", op); // the critical section
    // TODO: unblocking -> unblock here

    break;
}

```

Put one call on each TODO line. Do not move them.

**Every block must be matched by an unblock on every path**

If you block *above* the “Please login first!” check, that path *breaks* out of the case without reaching your unblock. `SIGINT` then stays blocked for the rest of the run, and **Ctrl+C stops working entirely**, silently. The TODO positions avoid this because all the early *breaks* come before the block. Keep the calls exactly where the TODOs are.

**The count is textual, so placement is on you**

The grader can only count calls, because a correctly deferred `SIGINT` changes nothing visible in a scripted session. Missing one gives `client.cpp` calls `block_signals` 6 time(s) and `unblock_signals` 7 time(s); each of the 7 transactions needs one of each, around the call that talks to the server. But calls in the *wrong* place still pass the counter. A TA reading your code checks placement.

**Ctrl+C at “Retry? (y/n)” waits for your answer**

The retry prompt is printed by `retry_operation`, which is *inside* the protected region. If you press Ctrl+C there, nothing happens until you answer `y` or `n`. Then the unblock delivers the signal and the client shuts down gracefully. That is correct. We tested it: even two presses at that prompt give one graceful shutdown, because a blocked signal does not queue (Sec. 2.2).

**What exactly is being protected**

Look inside the `deposit` lambda. It sends `DEPOSIT` to `finance`, reads the new balance, *then* sends an audit record to `logging`. Without blocking, a Ctrl+C between those two sends would let the menu loop exit and start the shutdown: money moved, nothing logged. With blocking, the `SIGINT` waits until `retry_operation` returns, both messages are done, and `unblock_signals()` delivers it.

You can see it in `signals.log`: a deferred Ctrl+C prints its handler line *between* the block line and the unblock line, because the handler runs inside `sigprocmask(SIG_UNBLOCK)`, before the unblock message is logged:

```
2026-10-08 02:19:39 - Signals blocked for critical section
SIGINT received - initiating graceful shutdown
2026-10-08 02:19:41 - Signals unblocked
2026-10-08 02:19:41 - Beginning graceful shutdown
```

**Where you will use this for real**

- ATMs and payment terminals are built so that a power cut or cancel button cannot land between “debit the account” and “record the debit”.
- Database clients wrap multi-statement updates in `BEGIN/COMMIT`. This is the same idea, enforced by the server instead of by masking signals.
- Any program that writes a file in place (editors, package managers, `git`) has a window where an interruption leaves a half-written file. Deferring signals, or writing to a temporary file and renaming it, closes that window.

### The intuition

The shutdown is not dangerous in itself. It is dangerous only if it starts in the middle of something. Block/unblock is a “do not open” sign hung on the door for the few milliseconds the transaction takes. Anyone who knocks is let in as soon as the sign comes down.

### Takeaway

Fourteen calls: `block_signals()` right before and `unblock_signals()` right after each of the seven `retry_operation(...)` calls, exactly at the TODOs, after the early breaks. A Ctrl+C during a transaction is deferred to the unblock, never lost and never in the middle.

### Done

```
lab-4 local tests: 100/100
Atomic flags declared and initialised 5/5
Handlers installed with sigaction 15/15
...
All three servers registered with the signal handlers 5/5
Signals blocked and unblocked around every transaction 10/10
```

100 is necessary but not sufficient. The grader cannot see four of the bugs in this guide. Spend ten minutes on Chapter 5 before you submit.

## Prove It, Then Submit

`make test` raises signals inside small test programs. It never presses Ctrl+C on the real client. This chapter does. Each experiment takes about a minute, and each one shows a behaviour you built. Run them in your VM with **two terminals** open in the repository directory (in VS Code, the + button in the terminal panel opens a second one).

Your messages may differ from ours

The outputs below come from a reference build whose handlers write to **stdout** and `signals.log`. If yours write only to the log or only to stderr, the handler lines appear in a different place. That is fine. The behaviour is what matters.

### 5.1 Experiment 1: a normal session

```
make && ./client
```

Answer the setup prompts (1, `system.log`, 1, `.txt`), then log in as user 1, deposit 500, choose option 8, log out, and exit with 0. Afterwards, `cat signals.log`:

```
2026-10-08 02:32:39 - Signal handlers initialized
2026-10-08 02:32:39 - Client started
2026-10-08 02:32:39 - Registered server: finance (PID: 3525)
2026-10-08 02:32:39 - Registered server: logging (PID: 3526)
2026-10-08 02:32:39 - Registered server: file (PID: 3527)
2026-10-08 02:32:40 - Signals blocked for critical section
2026-10-08 02:32:40 - Signals unblocked
... one blocked/unblocked pair per transaction ...
2026-10-08 02:32:40 - Normal exit requested
2026-10-08 02:32:40 - Sending QUIT to all servers
2026-10-08 02:32:40 - QUIT sent to finance server
Child process terminated, PID: 3525
Child process terminated, PID: 3527
2026-10-08 02:32:40 - QUIT sent to file server
Child process terminated, PID: 3526
2026-10-08 02:32:40 - QUIT sent to logging server
2026-10-08 02:32:40 - Waiting for child processes
2026-10-08 02:32:40 - All child processes terminated
2026-10-08 02:32:40 - Client shutdown complete
```

handler lines have no timestamp, and that is correct

The highlighted lines come from your `SIGCHLD` handler. They have no timestamp because formatting a time needs `localtime/strftime`, which are not `async-signal-safe`. A timestamp on a handler line usually means the handler called `log_signal_event()`, which is the bug from Sec. 2.4. They can also appear slightly out of order relative to the “QUIT sent” lines, because a server can exit and trigger `SIGCHLD` before the client logs that it sent the QUIT.

## 5.2 Experiment 2: Ctrl+C at the menu (graceful shutdown)

Start the client, finish the setup prompts, and press **Ctrl+C** once at `Enter choice:.`

```
Enter choice: ^CSIGINT received - initiating graceful shutdown

Performing graceful shutdown...
Sending shutdown signals to servers...
Child process terminated, PID: 527
Child process terminated, PID: 529
Waiting for all child processes to terminate...
Shutdown complete.
```

Usually you will see three “Child process terminated” lines. The run above happened to show two (see the note below). Then, in the second terminal:

```
pgrep -l -x 'finance|logging|file' # expect: no output
ls fifo_* # expect: No such file or directory
```

### Ctrl+C prints a line but the client keeps waiting

If the handler message appears but the menu does not shut down until you press Enter, `SIGINT` has `SA_RESTART`, almost certainly because of `signal()` instead of `sigaction`. See Task 2.

### Only two “Child process terminated” lines?

That is a race in the provided shutdown code, not your bug. After sending QUIT, `main` calls `while (wait(NULL) > 0);`. Sometimes `wait()` reaps a server before your `SIGCHLD` handler gets to it. The handler’s `waitpid` then finds nothing for that server, so no line is printed and its registry entry stays `ACTIVE`. The process is still reaped, and nothing is left running. It is occasional. In our testing it happened in about one Ctrl+C shutdown in ten, and in none of twelve normal exits.

## 5.3 Experiment 3: Ctrl+C during a transaction (deferred)

You need a transaction that pauses inside the protected region. A failed withdrawal does that, because the retry prompt is inside `retry_operation`:

1. Log in as user 1 (balance 0), choose 3, and withdraw 999.
2. At `Operation failed. Retry? (y/n):`, press **Ctrl+C**, even twice. **Nothing happens.** `SIGINT` is blocked and pending.
3. Type `n` and press Enter. `unblock_signals()` runs, the held signal is delivered, and the client shuts down gracefully.



```

Withdrawal failed: Insufficient funds
Operation failed. Retry? (y/n): ^C^Cn
Operation canceled.
SIGINT received - initiating graceful shutdown

Performing graceful shutdown...
Shutdown complete.

```

Notice that two presses produced **one** “SIGINT received” and a graceful shutdown, not a forced exit. A blocked signal is one pending bit, so the second press has nowhere to go. In `signals.log` the handler line sits between Signals blocked for critical section and Signals unblocked (Task 9).

## 5.4 Experiment 4: the second Ctrl+C (forced exit)

At the menu, two quick presses usually merge into one (graceful shutdown takes under 3 ms in this program, so there is no time to land a second press). To see the forced exit, make the shutdown get *stuck*:

1. Terminal 1: start the client and finish setup.
2. Terminal 2: freeze the finance server.

```

pgrep -l -x finance # e.g. 3938 finance
kill -STOP 3938 # finance stops running (it does not die)

```

3. Terminal 1: press Ctrl+C. The shutdown starts and hangs while sending QUIT to the frozen server.
4. Terminal 1: press Ctrl+C again.

```

Enter choice: ^CSIGINT received - initiating graceful shutdown

Performing graceful shutdown...
Sending shutdown signals to servers...
^CSecond SIGINT received - forcing exit
$ echo $?
130

```

The cost of forcing: the client did not stop its servers. All three are still running, and all six FIFOs are still on disk. Clean up from terminal 2:

```

kill -CONT 3938 # un-freeze first: a stopped process
                  # does not act on SIGTERM until continued
pkill -x finance; pkill -x logging; pkill -x file
make clean # also removes stale fifo_* files

```

### Why pkill -x, one name at a time?

`-x` matches the exact process name. Without it, `pkill file` would also match any process with “file” in its name. `pkill -f`, which matches the whole command line, is worse: it can match your editor, the grader, or your shell.

## 5.5 Things that look wrong but are not

| What you see                                                                       | Why it is expected                                                                    |
|------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Handler lines in <code>signals.log</code> have no timestamp                        | Timestamps need non-signal-safe functions (Sec. 2.4).                                 |
| Only 2 of 3 “Child process terminated” lines                                       | <code>main</code> ’s <code>wait(NULL)</code> sometimes reaps first (Experiment 2).    |
| Read failed: Interrupted system call                                               | A timeout working: <code>SIGALRM</code> interrupted the <code>read()</code> (Task 4). |
| “timed out after 10 seconds” after 30 seconds                                      | One alarm per process; <code>send_request</code> re-arms it to 30 (Task 7).           |
| Two Ctrl+C presses, but a graceful shutdown                                        | Signals of one type do not queue (Experiments 3 and 4).                               |
| Ctrl+C at “Retry? (y/n)” waits for an answer                                       | The prompt is inside the protected region (Task 9).                                   |
| <code>make test</code> reports the session did not complete when run twice at once | The grader’s <code>pkill -x client</code> kills the other run. Run one at a time.     |

## 5.6 Troubleshooting by grader message

Every message below was produced by running the grader on real broken code.

| Message (signals.cpp rows)                                                                 | Cause                                                                                             | Fix                                  |
|--------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|--------------------------------------|
| signals.cpp does not define what signals.h declares – undefined reference to ...           | Flags not defined, or defined outside the namespace                                               | Task 1                               |
| use of deleted function 'std::atomic<bool>::atomic(...)'                                   | = false on an atomic under C++11                                                                  | Use (false) or {false}               |
| SIGINT was installed with SA_RESTART; the task asks for default flags                      | signal() used, or flags carried over                                                              | sigaction with sa_flags = 0          |
| SIGCHLD is installed but without SA_RESTART                                                | Flags not set for SIGCHLD                                                                         | sa_flags = SA_RESTART                |
| SIGCHLD is handled by SIG_DFL (nothing was installed), not by SignalHandling::CHLD_handler | That signal's sigaction never ran (CHLD_handler means sigchld_handler)                            | Task 2                               |
| SIGINT or SIGALRM row says only no output                                                  | No handler installed, so the signal's <i>default action</i> killed the test before it could print | Task 2 first, then 3/4               |
| the SIGINT (or SIGALRM, SIGCHLD) handler recorded nothing                                  | Handler installed but writes nothing (it may also miss its flag; see the tip in Sec. 1.3)         | Tasks 3–5                            |
| after one SIGINT, shutdown_requested is still false                                        | Handler writes but does not set the flag                                                          | Task 3                               |
| the child exited but 'probe' is still marked active                                        | Registry entry not updated                                                                        | Set active = false on match (Task 5) |
| the child was still unreaped; use waitpid(..., WNOHANG) inside the handler                 | No SIGCHLD handler is running at all                                                              | Task 2                               |
| after block_signals(), SIGINT is not in the process mask                                   | sigprocmask missing or wrong set                                                                  | Task 6                               |
| after unblock_signals(), SIGINT is still blocked                                           | SIG_BLOCK used in unblock                                                                         | SIG_UNBLOCK                          |
| wait_with_timeout() left timeout_occurred set                                              | Flag reset removed or moved after the alarm                                                       | Reset, then alarm() (Task 7)         |
| no alarm was pending after wait_with_timeout(5)                                            | alarm() never called                                                                              | Task 7                               |
| cancel_timeout() left N seconds on the clock                                               | alarm(0) missing                                                                                  | Task 7                               |

| Message (client.cpp rows)                    | Cause                                              | Fix                                           |
|----------------------------------------------|----------------------------------------------------|-----------------------------------------------|
| no server was registered                     | Calls missing or inside the child block            | Task 8                                        |
| only file, logging registered                | A name misspelled or a call missing                | Exact lowercase names                         |
| client.cpp calls block_signals 6 time(s) ... | One transaction left unprotected                   | Task 9                                        |
| the session did not complete                 | Client crashed or hung, or two grader runs at once | Run ./client by hand; one make test at a time |

## 5.7 Before you submit

### The grader checks these

- ☐ make test shows **100/100**.
- ☐ make builds with no warnings (-Wall is on).

### The grader does NOT check these. A TA reading your code does.

- ☐ No handler calls log\_signal\_event, cout, printf, perror, std::string, to\_string or exit. Handlers use write/open/close/\_exit only.
- ☐ The second SIGINT really exits. Run the probe from Task 3, or Experiment 4.
- ☐ The file server is registered with file\_pid, and option 8 shows three *different* PIDs.
- ☐ Each block\_signals()/unblock\_signals() pair sits exactly at its TODOs, around retry\_operation.
- ☐ Experiments 2 and 3 behave as shown.

### Commit only your two files

```
make clean
git status # should list only signals.cpp and client.cpp as modified
git add signals.cpp client.cpp
git commit -m "Lab 4: signal handling"
git push
```

The repository's .gitignore already excludes binaries, \*.o, fifo\_\*, logs and storage/. Do not commit probe.cpp. Every push to main runs the autograder. Check the result on GitHub, and follow the handout for the official submission steps.

## 5.8 Quick reference

| Call                                                         | What it does                                       | In a handler? |
|--------------------------------------------------------------|----------------------------------------------------|---------------|
| <code>sigaction(sig, &amp;sa, NULL)</code>                   | install a handler with flags                       | —             |
| <code>sigemptyset / sigaddset</code>                         | build a signal set                                 | yes           |
| <code>sigprocmask(SIG_BLOCK, &amp;s, NULL)</code>            | add <code>s</code> to the blocked mask             | yes           |
| <code>sigprocmask(SIG_UNBLOCK, &amp;s, NULL)</code>          | remove <code>s</code> ; pending ones delivered now | yes           |
| <code>alarm(n) / alarm(0)</code>                             | arm (replacing any old alarm) / cancel             | yes           |
| <code>waitpid(-1, &amp;st, WNOHANG)</code>                   | reap any dead child, never block                   | yes           |
| <code>write(fd, buf, len)</code>                             | raw output                                         | <b>yes</b>    |
| <code>_exit(status)</code>                                   | terminate now, no cleanup                          | <b>yes</b>    |
| <code>exit, printf, cout, new</code>                         |                                                    | <b>no</b>     |
| <code>log_signal_event()</code>                              | timestamped log line (normal code only)            | <b>no</b>     |
| <code>raise(sig) / kill(pid, sig)</code>                     | send a signal to self / another process            | yes           |
| <code>kill -STOP &lt;pid&gt; / kill -CONT &lt;pid&gt;</code> | freeze / resume a process (shell)                  |               |
| <code>pgrep -l -x 'finance logging file'</code>              | list running servers (shell)                       |               |